Section 11: LangGraph Fundamentals

Graph-Based Orchestration for Stateful Agents

LangChain v1.2.x

March 2026

What We'll Cover

- Mental model: LangGraph as a graph-based orchestration framework
- LangGraph vs LangChain: relationships and use cases
- Core concepts: StateGraph, nodes, edges, START/END
- Building stateful, looping agents with cycles
- Checkpointing and persistence for resilience
- Human-in-the-loop interrupts and resumption
- Streaming, subgraphs, and dynamic graphs
- Patterns, debugging, common pitfalls

Goal: Master graph-based orchestration for complex agent workflows.

Mental Model: LangGraph as Graph Orchestration

- **LangGraph:** A graph-based framework for orchestrating multi-step agent workflows
- Nodes = functions that read/write *state*
- Edges = transitions between nodes (conditional or fixed)
- **State:** Shared data structure (TypedDict) passed through the graph
- Graph *runs* in discrete steps: execute a node, update state, route to next node
- **Stateful:** Unlike chains, graphs maintain and evolve state throughout execution
- **Looping:** Supports cycles, enabling iterative reasoning and refinement

Agents as Graphs: think in terms of state transitions, not function composition

Why LangGraph? Problems It Solves

- **Cycles & Loops:** Chains are DAGs (acyclic). Agents need feedback loops for reflection, retries, iteration.
- **Explicit State:** Chains pass one output as next input. LangGraph manages rich, multi-field state.
- **Conditional Routing:** Complex branching logic needs explicit graph structure, not nested if/else.
- **Persistence & Recovery:** Checkpointing state at each step enables interrupts, resumption, and audit trails.
- **Human-in-the-Loop:** Pause execution at specific nodes, let humans review/modify, resume seamlessly.
- **Composability:** Subgraphs enable modular agent designs.

LangGraph vs LangChain

Feature	LangChain	LangGraph
Structure	DAG (Acyclic Chain)	General Graph (with cycles)
State Management	Linear pass-through	Explicit TypedDict
Loops	Not directly supported	First-class citizen
Conditional Routes	Limited	Full branching/conditions
Checkpointing	Basic	Rich, fine-grained
Human Interrupts	No	Yes (interrupt_before, interrupt_after)
Ideal For	Simple pipelines	Complex agents, multi-step workflows

LangChain for chains; LangGraph for agents.

Core Concept: StateGraph

- **StateGraph:** A class that builds a graph with explicit state
- State is defined as a `TypedDict` (modern Python 3.8+)
- StateGraph constructor takes the state class
- You add nodes, edges, and configure entrypoint

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph

class AgentState(TypedDict):
    input: str
    output: str
    messages: list

graph = StateGraph(AgentState)
# Add nodes, edges, compile...
```

Key: State is the communication medium between nodes.

Nodes: Functions That Read and Write State

- **Node:** A Python function that receives state, performs work, returns updated state
- Signature: `def node_name(state: StateType) -> dict`
- **Return value:** A dict with keys to update in state (merged, not replaced)
- Nodes can call LLMs, tools, compute, or any Python code

```python
def process_node(state: AgentState) -> dict:
    """A node that processes the input."""
    result = state["input"].upper()  # Read state
    return {"output": result}  # Write state (merged)

# Add node to graph
graph.add_node("process", process_node)
```

Nodes are the "verbs" of your agent graph.

Edges: Connecting Nodes

- **Edge:** A transition from one node to another
- **Normal edge:** `add_edge(source, target)` — always routes from source to target
- **Conditional edge:** `add_conditional_edges(source, routing_func, mapping)`
- Routing function receives state, returns next node name

```python
# Normal edge: always go from "node_a" to "node_b"
graph.add_edge("node_a", "node_b")

# Conditional edge: routing function decides next node
def route(state: AgentState) -> str:
    if state["output"] == "DONE":
        return "end"
    return "loop"

graph.add_conditional_edges("process", route,
    {"end": END, "loop": "process"})
```

START and END: Special Nodes

- **START:** Virtual node representing graph entry point
- **END:** Virtual node representing graph exit
- Import: `from langgraph.graph import START, END`
- Always have at least one edge from START and one to END

```python
from langgraph.graph import START, END

# Route from START to first node
graph.add_edge(START, "input_node")

# Route to END when done
graph.add_edge("final_node", END)
```

Graph Structure

Every graph must have a clear entry point (START) and exit (END).

Building Your First Graph: Simple Chatbot

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class ChatState(TypedDict):
    user_input: str
    response: str

def chat_node(state: ChatState) -> dict:
    """Simple echo chatbot."""
    response = f"You said: {state['user_input']}"
    return {"response": response}

graph = StateGraph(ChatState)
graph.add_node("chat", chat_node)
graph.add_edge(START, "chat")
graph.add_edge("chat", END)

compiled = graph.compile()
result = compiled.invoke({"user_input": "Hello"})
print(result["response"])  # "You said: Hello"
```

State Management: How State Flows Through the Graph

- State is initialized when you call `invoke(initial_state)` or `stream(initial_state)`
- Each node receives the current state (all fields)
- Node returns a dict with updates; these are **merged** into state
- Merging: all keys in state persist unless overwritten
- After node completes, routing logic executes (conditional edges)
- Graph continues until it reaches END

State as Shared Memory

State is not replaced between nodes—it's accumulated. Each node updates only the fields it needs to.

Conditional Routing: add_conditional_edges

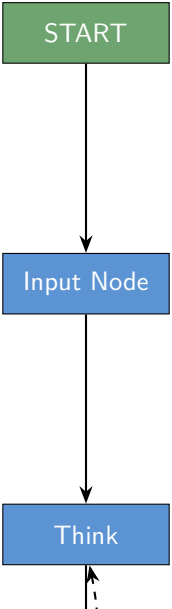
```
def routing_decision(state: ChatState) -> str:
    """Decide next node based on state."""
    if "error" in state.get("response", "").lower():
        return "retry"
    return "end"

graph.add_conditional_edges(
    "process_node",
    routing_decision,
    {
        "retry": "process_node",     # Loop back
        "end": END                    # Exit
    }
)
```

Key points:

- Routing function receives state, returns string (next node name)
- Mapping dict maps return value to actual node/END
- Enables loops, retries, branching based on runtime conditions

TikZ Diagram: Multi-Step Agent Graph with Branching



Cycles in Graphs: How LangGraph Enables Loops

- **LangChain chains:** DAGs (directed acyclic graphs) — no loops allowed
- **LangGraph:** Allows cycles — nodes can route back to earlier nodes
- **Loops enable:**
  - Iterative reasoning: "think -¿ act -¿ observe -¿ think again"
  - ReAct pattern: think, act, reflect, repeat
  - Error recovery: try -¿ fail -¿ retry
  - Refinement: generate -¿ evaluate -¿ improve
- Cycles are *explicit* — no hidden magic, you control when to loop

Infinite Loops

Always ensure your routing logic has an exit condition, or set a max recursion depth.

Checkpointing: Saving and Restoring Graph State

- **Checkpointing:** Saving state after each node executes
- **MemorySaver:** In-memory checkpoint storage (for development)

```python
from langgraph.checkpoint.memory import MemorySaver

# Create checkpointer
checkpointer = MemorySaver()

# Compile with checkpointer
compiled = graph.compile(checkpointer=checkpointer)

# Invoke with thread_id to enable checkpointing
result = compiled.invoke(
    {"user_input": "Hello"},
    config={"configurable": {"thread_id": "abc123"}}
)
```

Benefits:

- Resume execution from last checkpoint
- Inspect state at any step

Persistence Backends: SQLite and PostgreSQL

```python
from langgraph.checkpoint.sqlite import SqliteSaver

# SQLite persistence
checkpointer = SqliteSaver(conn="langgraph_db.sqlite")
compiled = graph.compile(checkpointer=checkpointer)
```

```python
from langgraph.checkpoint.postgres import PostgresSaver

# PostgreSQL persistence
checkpointer = PostgresSaver(
    conn_string="postgresql://user:pw@localhost:5432/db"
)
compiled = graph.compile(checkpointer=checkpointer)
```

Key points:

- SQLite: Good for single-machine, development
- PostgreSQL: For distributed, production systems
- Config: Always use `config={"configurable": {"thread_id": id}}`

Human-in-the-Loop: interrupt_before and interrupt_after

```
# Pause BEFORE executing a node
compiled = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["approval_node"]
)

# Pause AFTER executing a node
compiled = graph.compile(
    checkpointer=checkpointer,
    interrupt_after=["llm_call_node"]
)
```

Workflow:

1. Call `invoke()` or `stream()` — execution pauses at interrupt point
2. Check state: `compiled.get_state(config)`
3. Inspect results, let human decide
4. Optionally modify state
5. Resume: `compiled.invoke(None, config)`

Perfect for approval workflows, manual reviews, user feedback loops.

Resuming After Interrupts

```python
# First execution - pauses at "approval"
config = {"configurable": {"thread_id": "user_session"}}
compiled.invoke({"task": "process"}, config)

# Inspect state at interrupt point
state = compiled.get_state(config)
print(f"Pending input: {state.values}")

# Modify state if needed
updated_state = {
    "task": "process",
    "approved": True,
    "notes": "Looks good"
}

# Resume execution
result = compiled.invoke(updated_state, config)
```

State persists; execution picks up where it left off.

Streaming Graph Execution: stream() and astream_events()

- `invoke()`: Returns final state only
- `stream()`: Yields state after each node (useful for long operations)
- `astream_events()`: Async stream of intermediate events (for real-time updates)

```
# Stream states after each node
for snapshot in compiled.stream({"input": "..."}, config):
    node_name, node_state = snapshot
    print(f"Node '{node_name}' completed")
    print(f"State: {node_state}")

# Async streaming
async for event in compiled.astream_events(input, config):
    print(f"Event type: {event['event']}, data: {event['data']}")
```

Subgraphs: Composing Graphs Within Graphs

- **Subgraph:** A compiled graph used as a node in another graph
- Enables modular, reusable agent components
- Subgraph state must align with parent state (or use adapters)

```
# Define a sub-graph
sub_graph = StateGraph(AgentState).compile()

# Use it as a node in parent graph
parent_graph = StateGraph(AgentState)
parent_graph.add_node("subprocess", sub_graph)
parent_graph.add_edge(START, "subprocess")
parent_graph.add_edge("subprocess", END)

compiled = parent_graph.compile()
```

Use cases: Breaking complex agents into reusable modules, orchestrating teams of sub-agents.

Dynamic Graph Modification at Runtime

- **Note:** LangGraph graphs are typically compiled once
- For true dynamic routing, use conditional_edges with complex logic
- `dspy`, `guidance`, or similar tools can modify prompts dynamically
- Avoid rebuilding the entire graph on each invocation

Design for Flexibility

Use rich state and conditional edges to handle dynamic logic, rather than rebuilding graphs.

```python
# Conditional edges enable dynamic routing
def dynamic_router(state: AgentState) -> str:
    # Complex logic to decide next node
    if state.get("needs_tool_call"):
        return "tool_call_node"
    return "llm_node"

graph.add_conditional_edges("think", dynamic_router, {...})
```

Map-Reduce Patterns in LangGraph

- **Map:** Process multiple items in parallel (e.g., analyze documents)
- **Reduce:** Aggregate results (e.g., summarize)
- LangGraph supports concurrent node execution via state + conditional edges

```python
# State tracks batches
class BatchState(TypedDict):
    items: list
    results: list

def process_batch(state: BatchState) -> dict:
    """Process items (could use ThreadPoolExecutor)."""
    results = [process(item) for item in state["items"]]
    return {"results": results}

def aggregate(state: BatchState) -> dict:
    """Reduce: combine results."""
    final = sum(r["value"] for r in state["results"])
    return {"final_result": final}
```

Useful for parallel processing, batch analytics, multi-document workflows.

Error Handling and Retry in Graph Nodes

```python
import tenacity

@tenacity.retry(
    stop=tenacity.stop_after_attempt(3),
    wait=tenacity.wait_exponential(multiplier=1, min=2, max=10)
)
def flaky_node(state: AgentState) -> dict:
    """Node with automatic retry."""
    # This will retry up to 3 times on failure
    result = call_unstable_service(state["input"])
    return {"output": result}

# For more control, catch errors in conditional edge
def error_handler(state: AgentState) -> str:
    if "error" in state:
        return "retry"
    return "success"

graph.add_conditional_edges("call_api", error_handler, {...})
```

Combine retries with conditional routing for robust workflows.

Debugging LangGraph: Visualizing the Graph

```python
# Draw graph as ASCII or Mermaid
print(compiled.get_graph().draw_ascii())

# Or get as Mermaid diagram for visual tools
mermaid_str = compiled.get_graph().to_mermaid()
print(mermaid_str)

# Inspect nodes and edges
graph_obj = compiled.get_graph()
print("Nodes:", graph_obj.nodes)
print("Edges:", graph_obj.edges)
```

Debugging techniques:

- Use `stream()` to watch state evolution
- Add print statements in nodes
- Use `get_state()` to inspect at interrupts
- Check `get_graph().draw_ascii()` for structure validation

Debugging: Tracing and Introspection

```python
# Trace execution with debug config
config = {
    "configurable": {"thread_id": "debug_1"},
    "debug": True
}

# Stream with debug info
for step in compiled.stream(initial_state, config):
    print(f"Step: {step}")

# Get full state history (with checkpointer)
# Retrieve all checkpoints for a thread
states = compiled.get_state_history(config)
for state_snapshot in states:
    print(f"Checkpoint: {state_snapshot.checkpoint}")
    print(f"Values: {state_snapshot.values}")
```

Checkpointing gives you full audit trail of state evolution.

Common Pattern: ReAct Agent as a Graph

ReAct: Reasoning + Acting in a loop

```
class ReActState(TypedDict):
    question: str
    thoughts: str
    action: str
    observation: str
    answer: str

def think_node(state: ReActState) -> dict:
    thoughts = llm.invoke(f"Think about: {state['question']}")
    return {"thoughts": thoughts}

def act_node(state: ReActState) -> dict:
    action = llm.invoke(f"Given: {state['thoughts']}, what action?")
    obs = tools.invoke(action)
    return {"action": action, "observation": obs}

def should_continue(state: ReActState) -> str:
    if "ANSWER:" in state["observation"]:
        return "end"
    return "think"
```

Common Pattern: Multi-Agent Collaboration

```python
class CollabState(TypedDict):
    topic: str
    agent_a_response: str
    agent_b_response: str
    final_answer: str

def agent_a(state: CollabState) -> dict:
    resp = agent_a_llm.invoke(state["topic"])
    return {"agent_a_response": resp}

def agent_b(state: CollabState) -> dict:
    resp = agent_b_llm.invoke(state["topic"])
    return {"agent_b_response": resp}

def synthesize(state: CollabState) -> dict:
    combined = combine(state["agent_a_response"],
                       state["agent_b_response"])
    return {"final_answer": combined}

# Graph: START -> agent_a & agent_b (parallel) -> synthesize -> END
```

Use subgraphs or parallel execution for multi-agent workflows.

Performance Considerations

- **State size:** Keep state lean; avoid storing large objects (use references)
- **Checkpoint overhead:** Checkpointing after each node has cost; use in-memory for dev, persistent for prod
- **Streaming vs invoke:** Streaming is better for long runs; invoke waits for completion
- **Parallel nodes:** LangGraph doesn't parallelize by default; use Python `concurrent.futures` in nodes
- **Async:** Use async versions (`ainvoke`, `astream_events`) for I/O-heavy nodes
- **Recursion depth:** Monitor cycles to prevent runaway loops; set max iterations if needed

Optimization

Profile your graph with `stream()` and measure checkpoint latency in production.

Common Pitfall 1: State Mutation

Pitfall: Modifying state dict in-place leads to unexpected behavior.

```
# WRONG - mutates state in place
def bad_node(state: AgentState) -> dict:
    state["messages"].append("new message")  # Mutates!
    return {}  # Returns nothing, but state changed!

# CORRECT - returns new dict
def good_node(state: AgentState) -> dict:
    new_messages = state["messages"] + ["new message"]
    return {"messages": new_messages}
```

Rule: Always return a dict with fields to update; don't mutate the input state.

Common Pitfall 2: Missing Edges or Unreachable Nodes

Pitfall: A node has no outgoing edge, causing execution to hang.

```
# WRONG - "process" node has no outgoing edge
graph.add_node("process", process_fn)
graph.add_edge(START, "process")
# Missing: graph.add_edge("process", END)

# CORRECT - ensure all nodes have edges
graph.add_edge("process", END)
```

Checklist:

- START has at least one outgoing edge
- Every non-END node has at least one outgoing edge
- Every node is reachable from START
- Conditional edges cover all routing function outputs

Common Pitfall 3: Infinite Loops

Pitfall: Routing logic never reaches END.

```
# WRONG - always routes back to self
def bad_route(state: AgentState) -> str:
    return "loop_node"  # Never exits!

# CORRECT - has exit condition
def good_route(state: AgentState) -> str:
    iterations = state.get("iteration", 0)
    if iterations >= 3:
        return "end"
    return "loop_node"

graph.add_conditional_edges("loop_node", good_route, {
    "loop_node": "loop_node",
    "end": END
})
```

Prevention: Add counters, check for convergence, set max recursion limits.

Common Pitfall 4: State Type Mismatches

Pitfall: Nodes return types not matching TypedDict schema.

```
class MyState(TypedDict):
    count: int
    items: list

# WRONG - returns wrong type
def bad_node(state: MyState) -> dict:
    return {"count": "five"}  # Should be int!

# CORRECT - return correct types
def good_node(state: MyState) -> dict:
    return {"count": state["count"] + 1}
```

Tip: Use strict type checking or mypy to catch these errors early.

Cheat Sheet: Creating a Graph (1/2)

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

# 1. Define state
class AgentState(TypedDict):
    user_input: str
    output: str

# 2. Define nodes
def process_node(state: AgentState) -> dict:
    return {"output": state["user_input"].upper()}

# 3. Create graph
graph = StateGraph(AgentState)

# 4. Add nodes
graph.add_node("process", process_node)

# 5. Add edges
graph.add_edge(START, "process")
graph.add_edge("process", END)
```

Cheat Sheet: Compiling and Invoking (2/2)

```python
# 6. Compile
compiled = graph.compile(checkpointer=MemorySaver())

# 7. Invoke
result = compiled.invoke(
    {"user_input": "hello"},
    config={"configurable": {"thread_id": "1"}}
)
print(result["output"])

# 8. Stream (see intermediate steps)
for snapshot in compiled.stream(
    {"user_input": "hello"},
    config={"configurable": {"thread_id": "2"}}
):
    print(snapshot)

# 9. Get state at any point
state = compiled.get_state(config)
print(state.values)
```

Comparison: LangChain Chain vs LangGraph

LangChain Chain (simple linear):

```python
from langchain_core.runnables import RunnableSequence
chain = step1 | step2 | step3
result = chain.invoke({"input": "..."})
```

LangGraph (stateful, looping):

```python
graph = StateGraph(MyState)
graph.add_node("step1", step1)
graph.add_node("step2", step2)
graph.add_node("step3", step3)
graph.add_edge(START, "step1")
graph.add_conditional_edges("step2", route, {...})
compiled = graph.compile()
result = compiled.invoke({"input": "..."})
```

When to use:

- **Chain:** Simple linear pipelines
- **LangGraph:** Complex, looping, interactive agents

Integration with LangChain Tools

```python
from langchain_core.tools import tool
from langgraph.prebuilt.tool_executor import ToolExecutor

@tool
def search(query: str) -> str:
    """Search the web."""
    return f"Results for {query}"

tools = [search]
tool_executor = ToolExecutor(tools)

def tool_node(state: AgentState) -> dict:
    tool_calls = state["tool_calls"]
    results = [tool_executor.invoke(tc) for tc in tool_calls]
    return {"tool_results": results}

graph.add_node("tools", tool_node)
graph.add_edge("agent", "tools")
graph.add_edge("tools", "agent")
```

LangGraph + LangChain tools = powerful agent loops.

LLM Integration: Structured Output

```python
from pydantic import BaseModel
from langchain_core.language_models import BaseLanguageModel

class AgentAction(BaseModel):
    action: str
    input: str

def llm_node(state: AgentState) -> dict:
    # Use structured output from LLM
    llm = ChatOpenAI(model="gpt-4").with_structured_output(
        AgentAction
    )
    action = llm.invoke([...])
    return {
        "action": action.action,
        "input": action.input
    }
```

Ensure LLM outputs conform to your state schema.

Testing LangGraph Agents

```python
import pytest

def test_simple_agent():
    graph = create_agent_graph()
    compiled = graph.compile()

    result = compiled.invoke({
        "user_input": "Hello"
    })

    assert "output" in result
    assert result["output"] is not None

def test_conditional_routing():
    # Test that routing works
    result = compiled.invoke({
        "user_input": "URGENT",
        "priority": "high"
    })
    assert result["routed_to"] == "urgent_handler"
```

Unit test nodes independently, then test full graph flows.

Production Deployment Considerations

- **Checkpointing:** Use PostgreSQL or SQLite in production, not MemorySaver
- **Monitoring:** Log node execution times, errors, and state transitions
- **Versioning:** Version your graph definitions for reproducibility
- **Resource limits:** Set timeouts on LLM calls, tool execution
- **Async:** Use async nodes for I/O-bound operations
- **Error handling:** Implement retry logic and graceful degradation
- **State cleanup:** Archive old thread states to avoid DB bloat
- **API integration:** Expose graph via FastAPI, handle concurrent requests

LangGraph is production-ready; plan for scale and resilience.

Self-Check Questions (1/2)

1. What is the key difference between LangChain chains and LangGraph?
2. How is state represented in LangGraph? What's the modern pattern?
3. Explain the relationship between nodes, edges, and state.
4. What does a node function return, and how is that merged into state?
5. How do you create a loop in a LangGraph?
6. What is the purpose of checkpointing?
7. Describe a use case for `interrupt_before` vs `interrupt_after`.

Self-Check Questions (2/2)

8. What's the difference between `invoke()`, `stream()`, and `astream_events()`?
9. How would you implement the ReAct (Reasoning + Acting) pattern as a LangGraph?
10. What are three common pitfalls in LangGraph design, and how do you avoid them?

Answers should cover: graph structure, state management, cycles, checkpointing, streaming, and patterns.

Summary: LangGraph Fundamentals

- **LangGraph:** Graph-based orchestration for stateful, looping agents
- **Core components:** StateGraph, nodes, edges, START/END
- **State:** Shared dict passed and merged through nodes
- **Cycles:** First-class support for loops via conditional routing
- **Checkpointing:** Save state at each step for interrupts, resumption, persistence
- **Human-in-loop:** Pause execution, modify state, resume with interrupts
- **Streaming:** Watch state evolution with `stream()` for long runs
- **Patterns:** ReAct, multi-agent, map-reduce, tool-use loops
- **Production-ready:** Use SQLite/PostgreSQL checkpointing, async, error handling

Next: Apply LangGraph to build complex agents with reasoning, tool use, and human feedback!

Key Takeaways

Graphs $\neq$ Chains

State + Nodes + Edges + Conditional Routing =
Powerful, looping, interactive agents

Three Rules

1. Define state as TypedDict
2. Nodes return dicts (merge, don't replace)
3. Conditional routing enables loops

Master LangGraph and unlock agent complexity.

Additional Resources

- **LangGraph Docs:** `https://langgraph.dev`
- **StateGraph API:** TypedDict-based state definitions
- **Checkpointing:** MemorySaver, SqliteSaver, PostgresSaver
- **Prebuilt agents:** `create_react_agent()` in langgraph.prebuilt
- **Examples:** LangChain docs, LangGraph GitHub repo
- **Community:** LangChain Discord for patterns and troubleshooting

[Docs: langgraph.graph], [Docs: langgraph.checkpoint]

Thank You!

Questions?

Section 11: LangGraph Fundamentals
Graph-based orchestration for intelligent agents

Build, test, deploy, and iterate with LangGraph.